Unit - IV

Semantic Analysis:

→ The plain - parse - tree constructed in the syntax analyzer phase and it is no use for a compiler, as it does not carry any information of how to evaluate the tree.

Eg: $E \rightarrow E + T$

The above CFG production has no semantic rule associated with it, and it cannot help in making any sense of the production.

semantics: semantics of a language provide to its constructs, like tokens and syntax structure.

Syntax Directed Def^n = CFG + Semantic Rules

Attribute Grammar: Attribute Grammar is a special form of context - free grammar where some additional information (attributes) are appended to one (or) more of its non-terminals in order to provide context - sensitive information.

Each attribute has well-defined domain of values, such as integer, float, character, string & Expression.

Attribute grammar can pass values or information among the nodes of a tree.

Eg: $E \longrightarrow E + T$ $\{E.value = E.value + T.value\}$

The right part of the CFG contains the semantic rules that specify how the grammar should be interpreted. Here the values of non-terminals E and T are added together and the result is copied to the non-terminal E.

Attributes: Attributes are divided in to 2 types. They are

① synthesized attributes.

② Inherited attributes

① Synthesized attributes: These attributes get values from the attribute values of their child nodes.

$S \rightarrow ABC$

If S is taking values from its child nodes (A.B.C) then it is said to be a synthesized attribute. as the values of A.B.C are synthesized to S.

Ex: $A \rightarrow BCD$ A = Parent BCD = children

$A.val = B.val$
$A.val = C.val$
$A.val = D.val$

} Parent node A from its children B.C.D

② Inherited Attribute: In contrast to synthesized attribute, inherited attributes can take values from parent and/or siblings.

Ex:

$S \rightarrow ABC$

A can get values from S, B and C
B can get values from S, A and C
C can get values from S, A and B

Syntax Directed Translations (SDT):

→ In syntax Directed Translation the production ~~times~~ with semantic actions Embedded with production bodies (Actions are specified)

Eg: $E \rightarrow E+T$ { Print '+' }

→ The general approach to syntax-Directed Translation is to construct a Parse tree (or) syntax tree and compute the values of attributes of attributes at the nodes of the tree by visiting them in some order.

→ SDT involves passing information bottom-up and/or top-down to the Parse tree in form of attributes attached to the nodes.

→ SDT rules use.

1. Lexical values of nodes
2. constants
3. attributes associated with the non-terminals in their definitions

Eg: construct syntax Directed Translation for the given grammar

(2*3+4)

$E \rightarrow E+T$

$E \rightarrow T$

$T \rightarrow T*F$

$T \rightarrow F$

$F \rightarrow num.$

Solution:

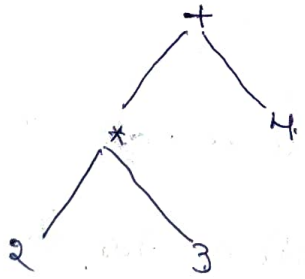
1. context free grammar + semantic actions are said to be syntax Definition Translation
2. Abstract syntax tree for 2*3+4
3. construct Parse tree for the above grammar
4. Annotated Parse Tree

step①: context free Grammar + semantic actions are said to be syntax Definition Translation.

Semantic actions are to be written in between curly braces.

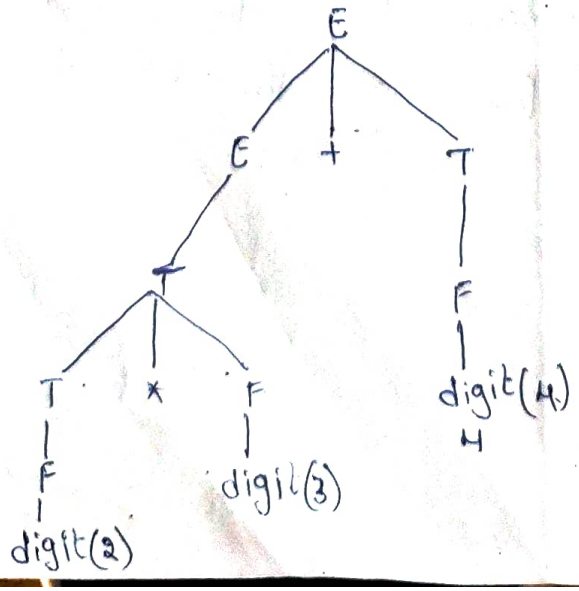
context Free Grammar	Semantic Actions
$E \rightarrow E+T$	{E.value = E.value + T.value}
$E \rightarrow T$	{E.value = T.value}
$T \rightarrow T*F$	{T.value = T.value * F.value}
$T \rightarrow F$	{T.value = F.value}
$F \rightarrow num.$	{F.value = num.lexical value}

step②: Abstract syntax tree for 2*3+4.

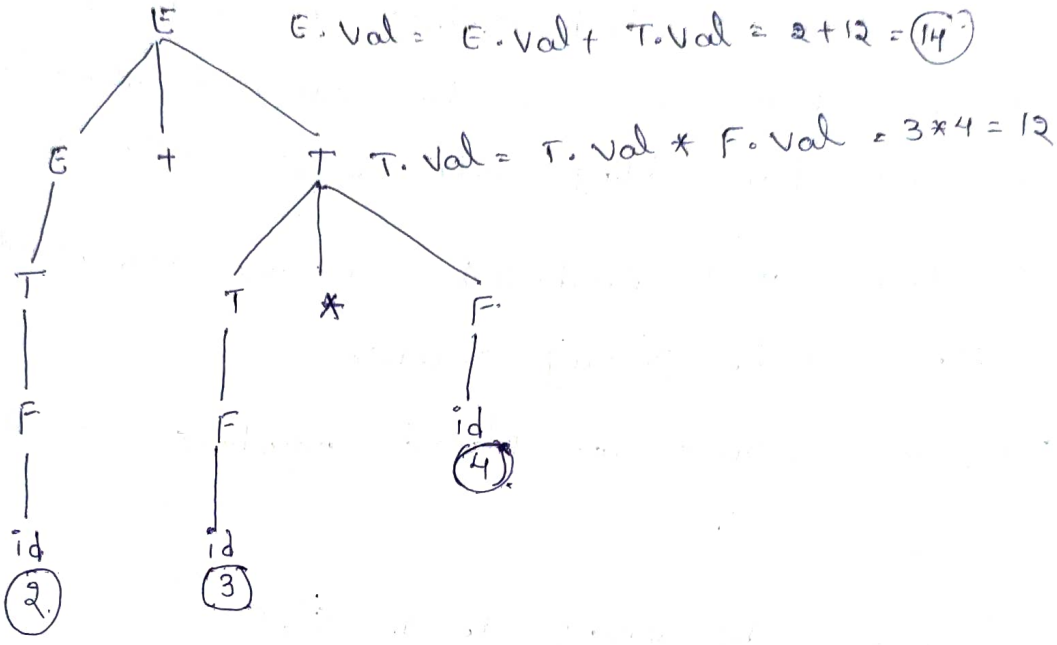


Abstract syntax tree

step③: construct parse tree for the above grammar



Step(4): Annotated Parse tree / Decorated Parse Tree



Types of SDT: There are 2 types of SDT

→ S-attributed SDT

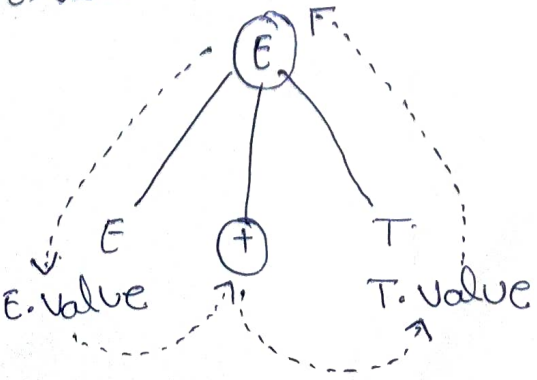
→ L-attributed SDT

S-attributed SDT: If an SDT uses only synthesized attributes, it is called as S-attributed SDT.

→ S-attributed SDTs are evaluated in bottom-up parsing, as the values of the parent nodes depend upon the values of the child nodes.

→ Semantic actions are placed in rightmost place of RHS

E.Value = E.value + T.value



L-attributed SDT :

→ If an SDT uses both synthesized attributes and inherited attributes with a restriction that inherited attribute can inherit values from left siblings only; it is called as L-attributed SDT.

→ Attributes in L-attributed SDTs are Evaluated by depth-first and left-to-right parsing manner.

→ semantic actions are placed anywhere in RHS.

Ex: $S \rightarrow ABC$

B can inherit the values from S & A
A can inherit the values from S
C can inherit the values from S, A, & B.

Difference between S-attributed SDT & L-attributed SDT

S-attributed SDT	L-attributed SDT
→ Based on synthesized attributes (i.e., get values from the attribute values of their children).	→ Based on synthesized and inherited attributes (i.e., can take values from parent and/or siblings.
→ use Bottom-up parsing	→ use Top-down parsing
→ semantic rule allways written at rightmost position in RHS	→ semantic rules anywhere in RHS.

① consider the grammar that is used for simple desk calculator. obtain the semantic action and also the annotated Parse tree for the string 3*5+4n

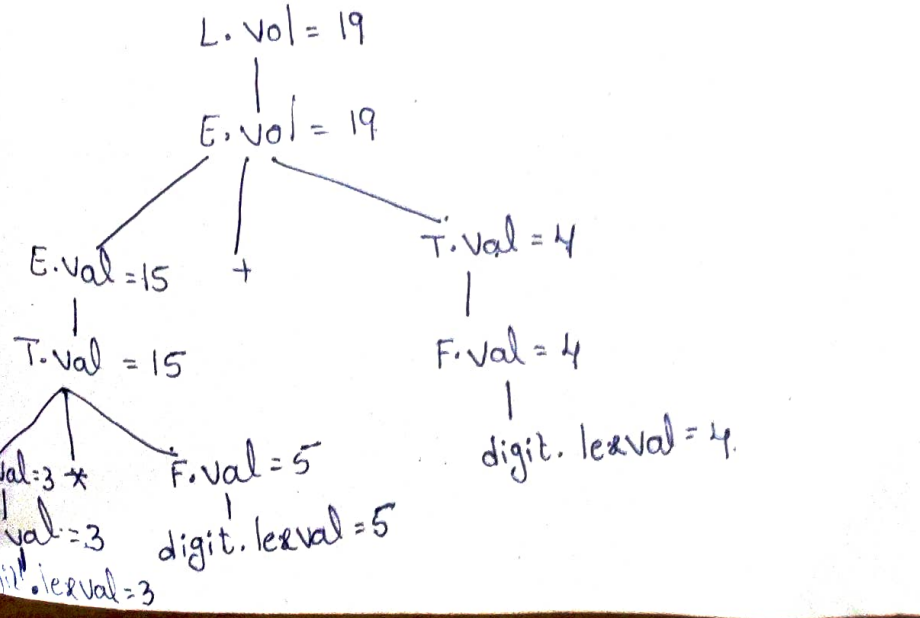
$L \to En$	$T \to T1 * F$
$E \to E1 + T$	$T \to F$
$E \to T$	$F \to (E)$
	$F \to$ digit

Sol: Syntax - Directed Definition for a simple desk calculator

Production	semantic Rule.
$L \to En$	$L.val = E.val$
$E \to E_1 + T$	$E.val = E_1.val + T.val$
$E \to T$	$E.val = T.val$
$T \to T1 * F$	$T.val = T_1.val * F.val$
$T \to F$	$T.val = F.val$
$F \to (E)$	$F.val = E.val$
$F \to$ digit	$F.val =$ digit.val

Syntax - directed definition for A simple desk calculator.

Annotated Parse Tree for 3*5+4n



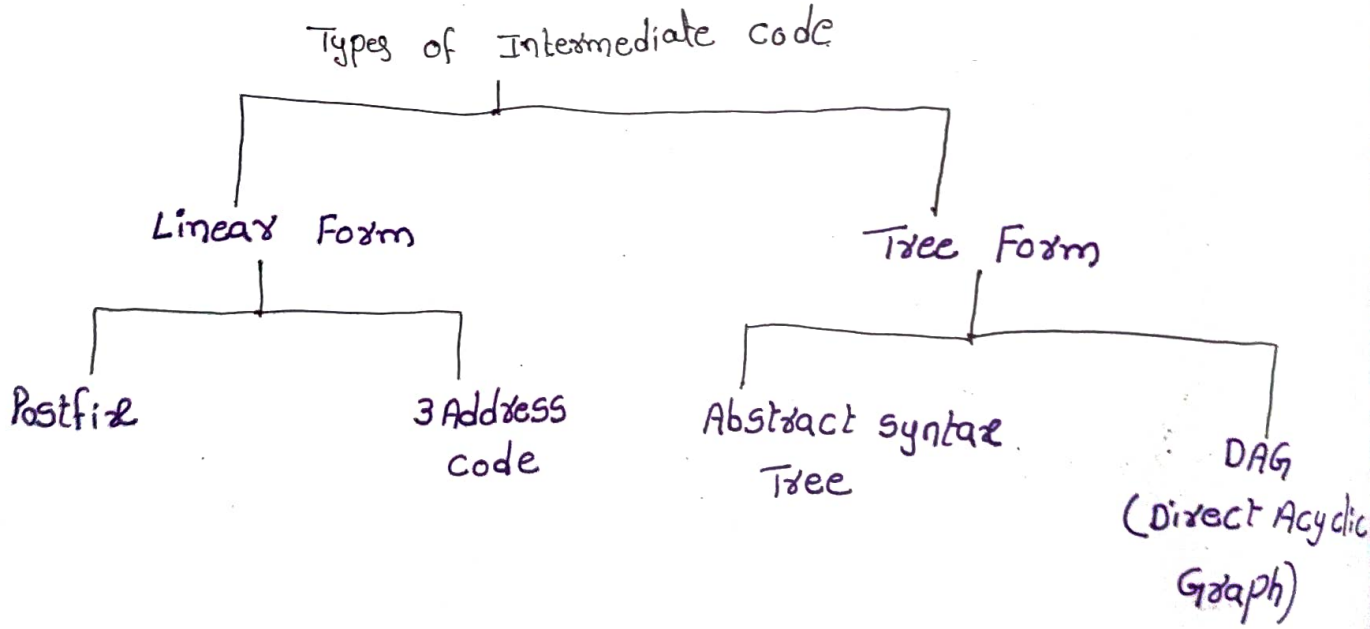
Intermediate code Generation

→ In the analysis - synthesis model of a compiler,

(i) the front end of a compiler translates a source program into an independent intermediate code.

iii) the back end of a compiler uses this intermediate code to generate the target code.

Types of Intermediate code:



① Postfix notation:

→ Also known as reverse polish notation (or) suffix notation

→ In this notation, the operator is placed after operands

Ex: ① a+b → Infix notation

ab+ → Postfix notation

② (a+b) *c ⟶ ab+c*

③ (a-b) * (c+d) + (a-b) ab-cd+*ab+

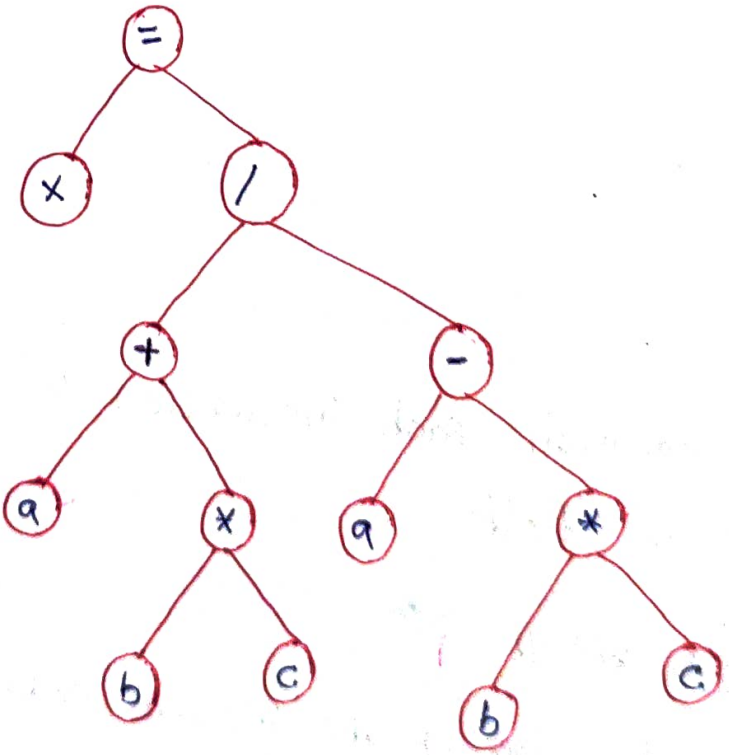
② Abstract syntax Tree:

→ Syntax tree is usually used when a represent a program in a tree structure.

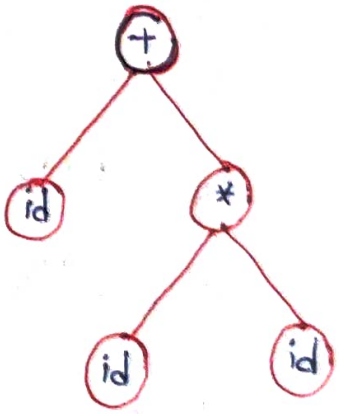
→ Each internal node represents operator

→ Each leaf node represents operand

Ex: ① $X = (a + (b*c)) / (a - (b*c))$



② id + id * id



③ Three Address code:

→ If any instruction is said to be three address instruction then the instruction satisfy the following conditions.

(i) Each instruction should contains at most 3 addresses

(ii) Right hand side at most one operator.

Implementation of Three Address code:

① Quadruple
② Triple
③ Indirect Triple

Sol:

① $x+y*z$ (not in three address)

$t_1 = x+y$;
$t_2 = t_1 * z$; } valid three address code representation.

② $a = b*-c + b*-c$.

$t_1 = -c$
$t_2 = b * t_1$
$t_3 = -c$
$t_4 = b * t_3$
$t_5 = t_2 + t_4$.

① Quadruple :

→ In quadruples representation, each instruction is splitted into the following 4 different fields.

op, arg1, arg2, result

→ The op field is used for storing the internal code of the operator.
→ The arg1 and arg2 fields are used for storing the two operands used.
→ The result field is used for storing the result of the expression.

	op	arg1	arg2	result
(0)	−	c		t_1
(1)	*	b	t_1	t_2
(2)	−	c		t_3
(3)	*	b	t_3	t_4
(4)	+	t_2	t_4	t_5

$a = b*-c + b*-c$
$t_1 = -c$
$t_2 = b * t_1$
$t_3 = -c$
$t_4 = b * t_3$
$t_5 = t_2 + t_4$

② Triple : The instruction is splitted into the following 3 different fields

op, arg1, arg2

→ op = operator

→ arg1, arg2 = arguments 1 and argument 2

	op	arg1	arg2
(0)	-	c	
(1)	*	b	(0)
(2)	-	c	
(3)	*	b	(2)
(4)	+	(1)	(3)

③ Indirect Triple Representation:

→ It uses an additional instruction array to list the pointers to the triples in the desired order.

→ Instead of position, pointers are used to store the results.

	Statement
35	(0)
36	(1)
37	(2)
38	(3)
39	(4)

	op	arg1	arg2
(0)	-	c	
(1)	*	b	(0)
(2)	-	c	
(3)	*	b	(2)
(4)	+	(1)	(3)

Advantages of Intermediate code Generation:

→ Easier to Implement: Intermediate code generation can simplify the code generation process by reducing the complexity of the input code, making it easier to implement.

→ Platform Independence: Intermediate code is platform-independent, meaning that it can be translated into machine code (or) byte code for any platform.

→ Code Reuse: Intermediate code can be reused in the future to generate code for other platforms (or) languages.

→ Easier Debugging: Intermediate code can be easier to debug than machine code (or) byte code, as it is closer to the original source code.

Disadvantages of Intermediate code Generation:

→ Increased compilation time: Intermediate code generation can significantly increase the compilation time, less suitable for real-time (or) time-critical applications.

→ Additional Memory usage: Intermediate code generation requires additional memory to store the intermediate representation.

→ Increased complexity: Intermediate code generation can increase the complexity of the compiler design, making it harder to implement and maintain.

→ Reduced Performance: The process of generating intermediate code can result in code that executes slower than code generated directly from the source code.

→ Platform Independence: Intermediate code is platform-independent meaning that it can be translated into machine code (or) byte code for any platform.

→ Code Reuse: Intermediate code can be reused in the future to generate code for other platforms (or) languages.

→ Easier Debugging: Intermediate code can be easier to debug than machine code (or) byte code, as it is closer to the original source code.

Disadvantages of Intermediate code Generation:

→ Increased compilation time: Intermediate code generation can significantly increase the compilation time, less suitable for real-time (or) time-critical applications.

→ Additional Memory usage: Intermediate code generation requires additional memory to store the intermediate representation.

→ Increased complexity: Intermediate code generation can increase the complexity of the compiler design, making it harder to implement and maintain.

→ Reduced Performance: The process of generating intermediate code can result in code that executes slower than code generated directly from the source code.